

CSCE 641 COMPUTER GRAPHICS · MODULE 06 · LECTURE NOTES

Light, Colour, and Local Shading

Why surfaces look the way they do, and the models that fake it.

MODULE OUTCOMES

Explain radiometric quantities and why irradiance falls off with cosine.

Derive Lambert's cosine law and implement diffuse shading.

Implement Blinn–Phong and state honestly where it is wrong.

Explain gamma and linear workflow, and why ignoring it looks bad.

Distinguish flat, Gouraud, and Phong interpolation and their artifacts.

1 Just enough radiometry

Shading models are easier to reason about, and much easier to debug, if you know which physical quantity each term is supposed to represent. Four quantities suffice for this course.

Quantity	Symbol	Units	What it is
Radiant flux	Φ	W	Total power carried by light.
Irradiance	E	W/m^2	Flux arriving per unit surface area. Depends on the angle the surface makes with the beam.
Radiance	L	$\text{W}/(\text{m}^2 \cdot \text{sr})$	Flux per unit projected area per unit solid angle. What a camera pixel and an eye both actually measure.
BRDF	f_r	1/sr	The fraction of radiance arriving from one direction that leaves in another. The complete description of a surface's appearance.

WHY RADIANCE

Radiance is invariant along a ray in a vacuum: the radiance leaving a surface toward the eye equals the radiance arriving at the eye from that surface, regardless of distance.

Irradiance is not invariant — it falls off with distance. Flux is not invariant — it depends on the area considered.

That invariance is precisely why 'what colour is this pixel' can be answered by computing something at the surface. Every renderer transports radiance.

2 Diffuse reflection

2.1 The cosine law, derived

Consider a parallel beam of light with cross-sectional area A striking a surface. If the surface is perpendicular to the beam, the power lands on area A . If the surface is tilted by θ , the same power is spread across $A/\cos\theta$ — a larger area — so the irradiance is reduced by $\cos\theta$.

$$\mathbf{E} = E_0 \cos\theta = E_0 (\mathbf{n} \cdot \mathbf{l})$$

This is worth internalising properly: the $\mathbf{n} \cdot \mathbf{l}$ factor appearing in every diffuse shader is not a property of the material and not an approximation of anything. It is the geometric foreshortening of the incident beam, and it would be there for any material whatsoever. The material's contribution is the BRDF, which for an ideal diffuse surface is a constant.

2.2 Energy conservation and the missing π

A Lambertian surface scatters incoming light equally in all directions over the hemisphere. For the total reflected energy not to exceed the incoming energy, the BRDF must be albedo/π , not albedo . The π arises from integrating the cosine over the hemisphere:

$$\int_{\Omega} \cos\theta \, d\omega = \pi$$

```
vec3 N = normalize(vNormal);
vec3 L = normalize(lightPos - vPos);
float ndotl = max(dot(N, L), 0.0);

vec3 diffuse = (albedo / PI) * lightColor * ndotl * attenuation;
```

A great deal of older code omits the $1/\pi$ and compensates by inflating light intensities. That is self-consistent but means your light units are arbitrary, which stops working as soon as you want physically based materials or image-based lighting. Keep the π .

3 Specular reflection

Diffuse reflection is the easy case because it is directionless. Specular reflection depends on the view direction, and the classical models for it are curve fits rather than derivations.

3.1 Phong and Blinn–Phong

	Phong (1975)	Blinn–Phong (1977)
Construction	Reflect the light direction about the normal, compare to the view.	Compute the halfway vector between light and view, compare to the normal.
Formula	$(\mathbf{R} \cdot \mathbf{V})^n$ where $\mathbf{R} = \text{reflect}(-\mathbf{L}, \mathbf{N})$	$(\mathbf{N} \cdot \mathbf{H})^n$ where $\mathbf{H} = \text{normalize}(\mathbf{L} + \mathbf{V})$
Cost	A <code>reflect()</code> per fragment.	A <code>normalize</code> of a sum. Cheaper.

	Phong (1975)	Blinn–Phong (1977)
Grazing angles	Highlight terminates abruptly.	Highlight stretches and elongates — which is what real surfaces do.
Verdict	Historical.	Still the right classical choice, and the halfway vector returns in Module 11 as the microfacet normal.

3.2 What is wrong with it

THREE REAL DEFECTS

No energy conservation. Increase the shininess exponent and total reflected energy changes arbitrarily. A surface can reflect more light than falls on it.

No Fresnel. Every real surface becomes highly reflective at grazing angles — look along a sheet of paper and it is nearly a mirror. Blinn–Phong has no mechanism for this, so edges look flat and materials look like plastic.

No physical parameters. 'Shininess 64' means nothing measurable, so materials must be retuned by hand whenever lighting changes. This is an enormous hidden cost in production.

Module 11 fixes all three with a microfacet model. Learn Blinn–Phong anyway: it is cheap, it is in all existing code, and the better model is easiest to understand as a correction of these specific failures.

4 Shading frequency

A shading model says what to compute. Shading frequency says how often, and the choice produces three recognisably different images.

Model	Evaluated	Interpolated	Result
Flat	Once per face	Nothing	Faceted appearance. Correct and desirable for genuinely flat surfaces; wrong for anything meant to look curved.
Gouraud	Once per vertex	The resulting colour	Smooth gradients, but specular highlights distort badly and can disappear entirely.
Phong	Once per fragment	The normal	Correct highlights at any triangle density. Costs one normalize and one shading evaluation per fragment. The default today.

Gouraud's failure mode is the most instructive. A specular highlight falls off as a high power of a cosine — exponent 128 is ordinary — which makes it a very narrow function over the surface. Gouraud samples that function at exactly three points per triangle. If the highlight's peak falls between the vertices, it is never sampled and the highlight simply does not appear. As the object rotates, highlights pop into and out of existence as the peak crosses vertex positions.

This is undersampling, which is the recurring theme of Module 01 appearing in a new place. The cure is to sample more often: evaluate per fragment instead of per vertex. Module 09 develops the general theory.

5 Gamma and the linear workflow

This section prevents more bad-looking renders than any other in the course, and the failure is insidious because the image does not look broken — it just looks wrong in a way that is hard to name.

5.1 The problem

Displays are not linear. An sRGB display emits luminance approximately proportional to the pixel value raised to 2.2. A value of 0.5 therefore emits about 21% of maximum brightness, not 50%. This curve exists for good reasons — human brightness perception is roughly logarithmic, so the encoding allocates more precision where the eye is more sensitive.

The consequence is that image files are stored in this non-linear encoding. A texture painted by an artist on a calibrated display is sRGB-encoded. If you feed those values directly into lighting arithmetic, you are multiplying and adding perceptual values as though they were quantities of light. They are not, and the results are wrong in specific, recognisable ways: midtones too dark, shadows muddy, highlights blown out, and blended edges with a dark fringe.

5.2 The fix

- 1 **Decode on input.** Convert sRGB textures to linear before use. Use an sRGB texture format and the hardware does it free, including correctly filtered.
- 2 **Compute in linear space.** All lighting, blending, and filtering happens on linear values.
- 3 **Encode once on output.** Convert back to sRGB at the very end. `glEnable(GL_FRAMEBUFFER_SRGB)` does this free.

THE TRAP THAT SURVIVES CODE REVIEW

Only **colour** textures are sRGB-encoded: albedo, emissive, and anything an artist painted as a colour.

Normal maps, roughness, metallic, ambient occlusion, height, and masks are **data**. They were never perceptual values, they were never encoded, and applying an sRGB decode to them corrupts them.

A normal map tagged sRGB produces lighting that is subtly wrong everywhere and obviously wrong nowhere — which is why this error survives into shipped products.

6 Attenuation

A point light radiates into a sphere. At distance d the energy is spread over $4\pi d^2$, so irradiance falls as $1/d^2$. This is exact physics, not a model.

$$\text{attenuation} = 1 / (d^2 + \epsilon)$$

The ϵ avoids division by zero at the light's exact position. Directional lights — the sun — have no falloff at all: they are infinitely distant, which in homogeneous terms means $w = 0$, exactly as Module 02 described.

Older engines frequently used linear or artist-tunable falloff curves, and the usual reason was that they were not gamma-correct: inverse-square looks far too harsh when applied to non-linear values. With a correct linear workflow, inverse-square is both correct and looks correct. If physically right falloff looks wrong in your renderer, suspect your gamma handling before you reach for a tunable curve.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
GAMES101 Lectures 07–09 — Shading	Lambert, Blinn–Phong, and the three shading frequencies, with the cosine-law derivation done geometrically.
John Hable — 'Why a Filmic Curve' and the gamma articles	The clearest practical writing on linear workflow and tonemapping, from someone who shipped it.
LearnOpenGL — Basic Lighting and Gamma Correction	The API mechanics of sRGB textures and framebuffers, with before/after images.

Resource	What it covers for this module
Naty Hoffman — 'Physics and Math of Shading' (SIGGRAPH course, free)	The best free introduction to radiometry for graphics. Read it before Module 11.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Add Lambertian diffuse shading to your software rasterizer with a single point light and correct inverse-square attenuation.
- 2 Implement Phong and Blinn–Phong side by side. Render a sphere lit at a grazing angle with both and capture the difference in highlight shape. Explain which is closer to a real surface and why.
- 3 Implement flat, Gouraud, and Phong shading on the same low-polygon sphere. Use a shininess exponent of 128 and capture all three. Find a camera angle where the Gouraud highlight disappears entirely.
- 4 Render the same scene with and without gamma correction. Then render a 50% grey gradient both ways and measure the emitted values. Write two sentences on why the uncorrected version looks the way it does.
- 5 Deliberately tag a normal map as sRGB and render. Describe the artifact precisely enough that you would recognise it in someone else's project.
- 6 Verify energy conservation numerically: integrate your diffuse BRDF over the hemisphere and confirm it equals the albedo. Then do the same for Blinn–Phong and observe that it does not.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why do renderers transport radiance rather than irradiance or flux?
- 2 Derive the cosine law from the geometry of a tilted surface, and explain why $\mathbf{n} \cdot \mathbf{l}$ is not a material property.
- 3 Where does the $1/\pi$ in the Lambertian BRDF come from, and what breaks if you omit it?
- 4 Give three specific defects of Blinn–Phong and say what each one looks like in a rendered image.
- 5 Why can a Gouraud-shaded specular highlight disappear completely? What is the general principle this illustrates?
- 6 State the three steps of a linear workflow, and list which texture types must not be sRGB-decoded.